

Ecosistema de orquestación de agentes para NodeFlow

Qué existe hoy, qué es verdad de lo que se dice, qué podés enchufar en esta máquina — y qué camino multiplica el crecimiento.

Preparado para **TOMAS.WAV (Tomas Pieruz)** · verificación de repos con `gh api` el 19/09/2026 · inventario por `which / --version` en la máquina

0 · Cómo leer esto (y por qué podés confiar en la lista de repos)

Este informe se apoya en tres tipos de afirmación, siempre marcados: verificado = lo comprobé en esta sesión (repos contra la API de GitHub con estrellas, licencia y fecha del último push; herramientas contra tu propia máquina); a probar = es plausible y está en la dirección correcta, pero depende de credenciales o de hardware que no puedo ejecutar por vos; descartado = circulaba en el material de investigación y **no existe o es irrelevante**.

La razón de esta sección: la primera pasada de investigación (tres agentes en paralelo, `gpt-oss:120b-cloud`) devolvió tablas impecables con **repositorios inventados**. Un informe que no separa lo verificado de lo narrado no sirve para decidir: sirve para sentirse productivo. Abajo está la lista de lo que se cayó, con la prueba.

descartado Lo que la investigación afirmó y no existe

Afirmación recibida	Realidad medida con <code>gh api</code>
<code>arthur-melton/crewAI</code> (★11k, Apache-2.0)	404: el repo no existe. El real es <code>crewAIInc/crewAI</code> — 58.762★, MIT.
<code>deepyard/agno</code> (★41k)	404. El real: <code>agno-agi/agno</code> — 42.245★, Apache-2.0.
<code>mikesquared-agency/axocoatl</code> — «runtime Rust con checkpointing»	404. No existe.
«Open-Agent-Delegation-Protocol (OADP)», estándar de delegación entre agentes, v0.2	El repo existe y tiene 0★ y último push 12/03/2026 . Un borrador que nadie usa no es un estándar.
<code>foobarto/glorbo</code> — «orquestador self-hosted»	Existe: 3★ . Es un experimento personal, no una pieza de infraestructura.
<code>spyrae/kronos-agent-os</code> — «runtime durable 24/7»	Existe: 51★ , MIT. No sostiene la recomendación que se le dio.
<code>Sathvik-1007/GraphMem-MCP</code> — «28 herramientas de memoria»	Existe: 1★ .
«VyasaGraph», «ClawGraph», «mem-graph (GoVanAI)», paper MAGMA <code>arXiv 2602.05665</code>	Sin rastro verificable (ni repo, ni paper con ese identificador).
Rendimiento de tu GPU «≈80 tok/s escalado desde una RTX 5080»	Metodología inventada (escalar por ancho de banda desde otro chip). Los números de la \$4 son los medibles.

«Tus modelos locales tienen la etiqueta <code>-cloud</code> »	Falso: los locales son <code>qwen2.5:7b</code> , <code>deepseek-r1:7b</code> , <code>granite3.3:2b</code> ; los <code>-cloud</code> son otros modelos servidos por Ollama.
NodeFlow « <code>github.com/maxnth/NodeFlow</code> »	Existe pero es otro proyecto (17★, sin actividad desde 06/2024). El tuyo es <code>NeonEden/NodeFlow</code> .

Regla que aplico de acá en adelante para cualquier recomendación de herramienta: **repo existente + licencia declarada + actividad en los últimos 6 meses + una superficie programática que se pueda llamar**. Sin las cuatro, no entra.

1 · Qué hay hoy en esta máquina verificado

1.1 Hierro

Componente	Detalle
GPU	AMD Radeon RX 6700 XT · 12 GB · driver 32.0.21045.5002
CPU	Ryzen 5 5600G (iGPU Vega integrada)
RAM	15,8 GB

El tope físico del ecosistema no es la GPU: son los 16 GB de RAM y **un solo modelo grande por vez** en la placa.

1.2 Runtime con CLI

Herramienta	Versión
opencode	1.18.18
aider	instalado (<code>~/.local/bin</code>)
Hermes Agent	perfil default, host-direct
Ollama / Docker	0.34.2 / 29.7.2
Node / Bun / Python	22.23.2 / 1.4.2 / 3.11.16
gh / az	— / 2.90.0 (200 USD de crédito)

1.3 Apps de agente instaladas — y el dato que cambia el diseño

Instaladas como **aplicaciones de ventana**: Antigraity (`Antigravity.exe`), Qwen (`Qwen.exe`), Claude Desktop, copilot, Cursor, Cline (extensión de VS Code, 4.1.19), Continue, Obsidian, Ollama.

Ninguna de esas cuatro grandes tiene CLI en el PATH
Claude, Qwen, Antigraity y Copilot no exponen binario de línea de comandos en tu máquina. Son *clientes* (y clientes MCP), no *trabajadores* llamables. Consecuencia directa: cualquier arquitectura que las trate como workers necesita un puente (MCP, automatización de UI o su API remota), y hoy ese puente no existe localmente. Lo que **sí** podés llamar por línea de comandos y capturar salida: opencode, aider, Hermes, Ollama, Docker, gh, az.

1.4 Modelos locales ya bajados

`qwen2.5:7b-instruct-q4_K_M` (4,7 GB) · `deepseek-r1:7b` (4,7 GB) · `granite3.3:2b` (1,5 GB). Vía Ollama en la nube: `kimi-k2.7-code` , `qwen3.5:397b` , `glm-5.3` , `gpt-oss:20b` , `deepseek-v4.1-flash` , `minimax-m3` , `nemotron-3` , `mistral-large-3` .

1.5 Lo que NodeFlow ya tiene construido

Pieza	Estado
App Tauri v2 (Rust + React), backend axum :37371	Instalada como programa (NSIS con marca, accesos directos, «Aplicaciones instaladas»)
Lienzo de nodos + grafo Kùzu + bóveda Obsidian	Funcional, sin publicar
Voz: STT (AssemblyAI/Speechmatics) + TTS local Kokoro :8125	Funcional; idioma de la voz desacoplado de la interfaz (19/09)
Servidor MCP propio (nodeflow_mcp.py) montado en Hermes	Funcional: Hermes ya opera el lienzo
Motores: DeepSeek, Gemini, Ollama (local y cloud), Foundry sin clave	Catálogo con disponibilidad declarada por motor
Investigación por fases con investigador + contraste	Cadena lista; bloqueada hoy por cuota de búsqueda (\$6.3)
Suite de tests	281 Rust + tsc limpio

1.6 Tres hechos medidos hoy que condicionan cualquier plan

1. **GitHub Models está retirado** (30/07/2026, changelog oficial): el endpoint contesta 410 Gone con un cuerpo que dice «temporarily unavailable... brownout». Cualquier rol apoyado en models.github.ai está muerto.
2. **La cuota de búsqueda de Gemini se agota antes que la del modelo:** con la herramienta de búsqueda todos los modelos devolvieron 429 RESOURCE_EXHAUSTED ; sin ella, el mismo modelo contestó 200 .
3. **El agente-CLI como buscador es lento:** una búsqueda delegada al runtime de agente pasó los 300 s y se cortó. Sirve como worker de código, no como buscador en camino caliente.

2 · El terreno real verificado

Todo lo de esta sección se consultó contra la API de GitHub el 19/09/2026: estrellas, licencia declarada y fecha del último push. La columna «activo» es el último push.

2.1 Protocolos: lo que hoy es un estándar de verdad

Protocolo	Qué resuelve	★	Lic.	Activo
MCP — <code>modelcontextprotocol/modelcontextprotocol</code>	Agente ↔ herramientas/recursos. Es el que ya usás (NodeFlow montado en Hermes, y opencode tiene <code>opencode mcp</code>).	9.254	MIT/Apache	19/09/26
ACP — <code>zed-industries/agent-client-protocol</code>	Editor ↔ agente: «conectar cualquier editor con cualquier agente». opencode lo implementa (<code>opencode acp</code>) .	4.282	Apache-2.0	19/09/26
AGENTS.md — <code>openai/agents.md</code>	Contrato de repo que leen los agentes: convenciones, comandos, reglas. Un archivo, efecto en todos los workers.	24.471	MIT	10/09/26
Agent Protocol — <code>langchain-ai/agent-protocol</code>	HTTP para servir agentes: hilos, corridas, streaming, estado.	677	MIT	18/09/26

Lectura: elegiste bien sin saberlo. **MCP es el bus** (y ya lo tenés montado) y **ACP es el enchufe de workers de código** (y tu opencode habla los dos). No hay que inventar protocolo: hay que usar el que tu herramienta más fuerte ya implementa.

2.2 Agentes-CLI: la fuerza de trabajo que ya se ejecuta sola

Herramienta	Qué es	★	Lic.	Activo
opencode — <code>sst/opencode</code>	El que tenés instalado. 208.572★ : es, por lejos, el proyecto más grande de la categoría.	208.572	MIT	19/09/26
Claude Code — <code>anthropics/claude-code</code>	El agente-CLI de Anthropic. Sin licencia declarada, con la comunidad más activa de todas.	146.574	—	19/09/26
Codex — <code>openai/codex</code>	Agente de código de OpenAI, terminal, Apache-2.0.	125.280	Apache-2.0	19/09/26
Gemini CLI — <code>google-gemini/gemini-cli</code>	Agente de terminal de Google, licencia permisiva.	107.085	Apache-2.0	19/09/26
OpenHands — <code>All-Hands-AI/OpenHands</code>	Agente de desarrollo completo, autohospedable.	88.511	MIT	19/09/26
Cline — <code>cline/cline</code>	El que tenés en VS Code; también existe como herramienta aparte.	68.760	Apache-2.0	19/09/26
Goose — <code>block/goose</code>	Agente extensible de Block: «más allá del código».	54.464	Apache-2.0	19/09/26
aider — <code>Aider-AI/aider</code>	El que tenés: edición por diff y commits automáticos.	49.057	Apache-2.0	22/05/26
Qwen Code — <code>QwenLM/qwen-code</code>	Agente de terminal de Qwen (encaja con tus modelos locales).	27.986	Apache-2.0	19/09/26
<code>opencode-ai/opencode</code>	El opencode anterior (sucesor: <code>sst/opencode</code>). Sin actividad desde 09/2025.	13.754	MIT	18/09/25

Lectura: el mercado convergió en «un agente que vive en la terminal, se llama por flag y devuelve texto o JSON». Eso es exactamente un *worker*. Tenés instalado el de mayor adopción (opencode) y uno de los más scriptables (aider).

2.3 Frameworks de orquestación: quién sigue vivo y con qué licencia

Framework	Nota	★	Lic.	Activo
crewAI	Equipos de agentes por roles; el más «de negocio».	58.762	MIT	19/09/26
LangGraph	Grafos de estado con checkpoints y human-in-the-loop.	41.935	MIT	18/09/26
agno	Agentes como herramientas; fuerte en evals.	42.245	Apache-2.0	18/09/26
openai-agents-python	SDK oficial, handoffs, tracing.	29.558	MIT	18/09/26
Semantic Kernel	Base .NET/Python de Microsoft.	28.578	MIT	19/09/26
microsoft/agent-framework	Su framework nuevo de agentes (13,6k★, un mes de vida).	13.597	MIT	18/09/26
AG2 (ex AutoGen)	La continuidad comunitaria de AutoGen.	4.940	Apache-2.0	19/09/26
AutoGen (original)	En mantenimiento: sin push desde 15/04/2026.	61.060	CC-BY-4.0	15/04/26
MetaGPT	Sin actividad desde 21/01/2026.	70.499	MIT	21/01/26
NVIDIA NeMo Agent Toolkit	Cliente y servidor MCP incluidos; nicho empresarial.	2.637	Apache-2.0	10/09/26

Lectura y advertencia: ninguno de estos frameworks vive en Rust ni piensa como una app de escritorio; meterlos en NodeFlow significaría un segundo runtime (Python) dentro de un producto Tauri. La tabla sirve para *copiar ideas* (checkpoints, handoffs, tracing), no para adoptar la dependencia.

2.4 Memoria de agentes con grafo: acá está el calor del mercado

Proyecto	Qué aporta	★	Lic.	Activo
mem0	Memoria persistente para agentes, multi-SDK.	65.637	Apache-2.0	19/09/26
Graphiti	Grafo temporal de conocimiento (bi-temporal) para agentes.	31.005	Apache-2.0	19/09/26
cognee	Pipeline de memoria desde documentos a grafo.	30.835	Apache-2.0	19/09/26
Letta (ex MemGPT)	Agentes con estado propio y memoria administrada.	24.797	Apache-2.0	10/09/26
Kùzu (lo que usa NodeFlow)	Grafo embebido con búsqueda vectorial. Sin push desde 10/10/2025.	4.024	MIT	10/10/25

Riesgo que no estábamos mirando: tu capa de grafo está sin actividad hace 11 meses

Kùzu es la pieza más «de fundación» de NodeFlow (el lienzo vive ahí) y su repositorio no recibe commits desde octubre de 2025. Puede ser un proyecto que terminó y está estable —o uno abandonado—; desde acá no se distingue. Lo importante es que **hoy no tiene plan B**. Antes de escribir más historia sobre Kùzu conviene decidir: exportar/migrar, encapsular el acceso en una capa única, o adoptar Graphiti/cognee (ambos muy activos) para la parte semántica.

2.5 Orquestadores self-hosted y ejecución duradera

Proyecto	Para qué	★	Lic.	Activo
n8n	Automatización visual con cientos de integraciones; el estándar de facto autohospedado.	205.352	s/d	19/09/26
Langflow	Construir y desplegar flujos de IA con UI.	155.003	MIT	19/09/26
Temporal	Ejecución duradera de verdad: los flujos sobreviven a caídas y reinicios.	23.170	MIT	19/09/26
Kestra	Orquestador de flujos declarativo (YAML), con UI.	28.168	Apache-2.0	18/09/26
Sim	Workspace colaborativo para construir y monitorear agentes.	29.675	Apache-2.0	19/09/26
Windmill	Scripts → flujos y colas, muy liviano (Rust + Svelte).	17.977	s/d	19/09/26
Restate	Ejecución duradera minimalista (Rust).	4.441	s/d	18/09/26
Dagu	Orquestador de un solo binario, sin dependencias (Go).	4.034	GPL-3.0	19/09/26

2.6 El resto del terreno que sirve a NodeFlow

Proyecto	Por qué te importa	★	Lic.
Ollama	Tu runtime local; API OpenAI-compatible y de herramientas.	181.264	MIT
markitdown (Microsoft)	Convertir cualquier archivo/Office a Markdown — ideal como herramienta MCP para el lienzo.	185.580	MIT
browser-use	Agentes que manejan el navegador de verdad (research, formularios, prospectos).	115.282	MIT
MCP servers (oficiales)	Catálogo de servidores listos para montar.	90.463	s/d
RAGFlow	Ingesta y RAG profundo sobre documentos.	90.990	Apache-2.0
tldraw	El SDK de lienzo infinito (tu dominio: canvas + nodos).	50.452	s/d
playwright-mcp	Navegador controlable por MCP: una herramienta, todos tus clientes.	37.329	Apache-2.0
Suna	«Sistema de gestión de IA» open source: referencia de producto, no de librería.	20.222	s/d

3 · Tus herramientas: qué superficie real exponen

verificado

Salida literal de `--help` en tu máquina. Esto es lo que se puede llamar hoy, sin instalar nada.

opencode 1.18.18 — el hallazgo más grande del relevamiento

No es «un TUI»: es un **servidor de agentes**. Subcomandos reales de `opencode --help`:

Subcomando	Qué te habilita
<code>opencode serve</code>	Servidor headless: otro proceso (Hermes, NodeFlow, un cron) le manda trabajo y recoge resultado.
<code>opencode acp</code>	Habla ACP (el estándar editor→agente, 4,3k★): cualquier cliente ACP puede manejarlo.
<code>opencode run [message..]</code>	Una tarea y salida: el worker más simple posible.
<code>opencode mcp</code>	Monta servidores MCP (tu <code>nodeflow_mcp.py</code> entra acá: opencode operaría el lienzo).
<code>opencode agent / plugin</code>	Define agentes y plugins propios.
<code>opencode export [sessionId] / import</code>	Sesión como JSON: resultado capturable y auditable.
<code>opencode stats</code>	Tokens y costo por uso: la métrica que hoy no tenés del lado de agentes.
<code>opencode github / pr</code>	Agente de GitHub y flujo de PR: cierra el ciclo hasta el commit.

aider — scriptable por diseño

`--message/-m` (una orden y sale), `--message-file`, `--exit`, `--yes-always`, `--apply FILE`, `--stream`, `--model`. No tiene API HTTP: se usa como proceso externo y se lee su salida. Ideal para «aplicá este diff y commitéalo».

Hermes — el orquestador que ya tenés (y que ya sabe más de lo que usás)

Subcomandos verificados: `chat`, `model`, `moa` (**Mixture of Agents**), `fallback` (cadena de proveedores), `worktree` (podar worktrees de git), `browser`, `secrets`, `egress` (firewall de egreso con inyección de credenciales), más `mcp`. Y en la línea base: `-z` (una orden y salir), `--resume` / `--continue` (sesiones), `--skills`, `--worktree`, `--yolo`. Sumado a lo que ya usás todos los días (subagentes en paralelo, cron, skills, cliente MCP, memoria), Hermes es el orquestador central: lo que faltaba no era un framework, era **conectarle workers y darle continuidad**.

Ollama, Docker, gh, az — los otros cuatro llamables

`ollama`: API HTTP en `:11434` (`/api/chat`, `/api/tags`, `/api/ps`) y `-cloud` para modelos grandes sin bajarlos. `docker`: aislamiento de workers y servicios. `gh`: PRs, issues, releases (y autenticado). `az`: Azure AI Foundry con 200 USD de crédito — el reemplazo natural del canal de Copilot que se murió.

4 · Teorías de continuación verificado a probar

Cada teoría tiene la misma forma: **tesis** (qué sostengo), **por qué** (la evidencia), **qué hace falta** (el trabajo concreto) y **cómo sé que funcionó** (la señal, medida). Están ordenadas por relación resultado/esfuerzo, no por elegancia.

Nivel A · multiplican ya, con lo que hay en la máquina

A1 Hermes orquesta, opencode trabaja (y el contrato es un archivo)

Tesis: el par orquestador/worker necesita un *contrato*, no una fusión. Hermes ya es el mejor orquestador disponible en la máquina (subagentes, cron, skills, MCP, sesiones, MoA, fallback); opencode 1.18.18 ya es un servidor de agentes (`serve` / `acp` / `run`). Falta lo barato: un `AGENTS.md` en el repo y una llamada.

Por qué: hoy el cuello de botella no es la capacidad de los modelos, es *mi contexto*: cada tarea de código que hago yo consume la ventana que necesito para pensar el sistema. Medido en esta misma sesión: tres subagentes investigaron en paralelo y devolvieron 6.000 palabras en 7 minutos. Eso ya es trabajo delegado; falta que el trabajo de *código* también lo sea.

Qué hace falta: `AGENTS.md` con comandos exactos (build, test, lint, formato) y reglas del repo; invocar `opencode run` (o `serve`) con tope de tiempo y *verificación obligatoria* por tests antes de aceptar el resultado.

Señal de éxito: una tarea real de NodeFlow (p. ej. «arreglar los 6 warnings de Rust y correr la suite») resuelta por un worker, con los 281 tests verdes y sin que yo escriba una línea.

A2 Un hub MCP compartido vale más que cinco agentes nuevos

Tesis: cada servidor MCP que montás sirve a *todos* tus clientes a la vez (Hermes, opencode, Cline, Claude Desktop, VS Code, NodeFlow). La inversión se multiplica por el número de clientes, no por el número de agentes.

Por qué: MCP (9,2k★, actividad diaria) es el único estándar con adopción real; ya lo usás con NodeFlow; y existen servidores de primer nivel listos: `playwright-mcp` (37k★) para el navegador, `markdown` (185k★) para documentos, el catálogo oficial (90k★).

Qué hace falta: montar 2-3 servidores (navegador, documentos, bóveda) una sola vez, y dejar de escribir integraciones ad-hoc dentro de la app.

Señal de éxito: la misma capacidad (p. ej. «leer un PDF y volcarlo al lienzo») disponible desde dos clientes distintos sin código nuevo.

A3 NodeFlow no debe ser el orquestador: debe ser la capa de intención y evidencia

Tesis: el camino equivocado es reimplementar LangGraph/Temporal dentro de Tauri. El correcto es el que ya empezaste: el lienzo guarda *qué se quiere* (nodos PENDIENTE/DECISIÓN/RIESGO/HITO), la bóveda guarda *por qué*, el MCP expone el lienzo, y los agentes (Hermes, opencode) ejecutan afuera.

Por qué: ningún framework de la §2.3 vive en Rust ni piensa como app de escritorio; meter uno es agregar un segundo runtime a un producto que ya tenés que empaquetar y firmar. Tu ventaja competitiva no es el ruteo de agentes (todos lo tienen): es el grafo + la bóveda + la voz + el panel de aprobación.

Qué hace falta: formalizar el contrato — «los agentes escriben planes en `bóveda/cerebro/planes/`, un worker los ejecuta, el lienzo registra el resultado». Y una regla dura: nada escribe en el lienzo sin pasar por la app.

Señal de éxito: una tarea delegada hoy que mañana se pueda explicar leyendo el lienzo, sin abrir el chat.

A4 La continuidad se resuelve con un job, no con más agentes

Tesis: el proyecto no se frena por falta de capacidad, se frena por falta de *retomar*: entre sesión y sesión se pierde el hilo. Un cron diario que condense `git log` + DEVLOG + los pendientes del lienzo en **tres movimientos** vale más que un agente más.

Por qué: ya tenés las piezas: cron de Hermes, `checkpoint.sh`, DEVLOG, nodos PENDIENTE, y los «pendientes elegidos» que anotamos hoy en `ROADMAP.md`. Falta la síntesis diaria.

Señal de éxito: abris la app y hay una lista de 3 cosas que hacer, sin releer el chat.

Nivel B · abren techo (30 a 90 días)

B1 Ejecución duradera: reanudar en vez de reintentar

Tesis: las tareas que valen (investigación completa, refactor grande, procesamiento del video demo) duran más que la paciencia de un proceso y se cortan. Hoy eso se resuelve a mano (topes de 300 s, banderas que caducan solas). El salto es un motor de ejecución duradera. **Por qué:** Temporal (23k★, MIT) es el estándar, pero pesado (servidor + workers); Restate (4,4k★, Rust) y Dagu (4k★, GPL-3.0, un binario) son proporcionados a tu escala.

Qué hace falta: elegir por peso, no por fama: **Dagu o Windmill en Docker** para los flujos de «muchos pasos con reintentos», dejando Temporal para el día que haya clientes. **Señal:** un flujo largo que se corta y retoma donde estaba, solo.

B2 Memoria: separar «recordar» de «buscar»

Tesis: tu memoria es rica (bóveda + grafo) pero no tiene capa de *recuperación*: qué recordar y cuándo. El mercado resolvió eso y está muy activo: Graphiti (31k★, grafo temporal bitemporal), cognee (30,8k★), mem0 (65,6k★). Con Kùzu sin actividad hace 11 meses, la pregunta es si la capa semántica conviene apoyarla en uno de estos. **Qué hace falta:** primero *encerrar* el acceso a Kùzu detrás de una interfaz propia (una sola puerta = migración barata después). **Señal:** poder cambiar el motor de grafo tocando un archivo.

B3 Sumar uno o dos agentes-CLI de terceros como especialistas

Tesis: no hace falta uno de cada uno; hace falta uno *distinto*. Hipótesis concreta: un segundo worker con otra familia de modelo (Gemini CLI 107k★, Qwen Code 28k★ — encaja con tus locales, Codex 125k★) reduce el sesgo compartido: cuando dos familias distintas coinciden, la probabilidad de error baja; cuando discrepan, aprendiste algo. **Señal:** un bug real encontrado por el worker B que el A no vio. **Cuidado:** cada CLI nuevo = otra cuota, otra factura, otro conjunto de permisos sobre tu repo.

B4 AGENTS.md: un archivo que entrena a todo worker futuro

Tesis: el formato AGENTS.md (24,4k★) es el contrato más barato del ecosistema: convenciones, comandos canónicos y «lo que no se toca». Escrito una vez, trabaja para opencode, Cline, Codex, Gemini CLI y cualquier agente que entre al repo. **Señal:** un worker nuevo produce un PR que respeta tus reglas sin que se las repitas.

B5 Las apps de ventana son consumidoras, no trabajadoras

Tesis: Antigravity, Claude Desktop y Qwen no tienen CLI (verificado): tratarlas como workers obliga a automatización de UI, que es frágil y rompe la máquina de estado de la app. Su rol correcto: **clientes MCP con humano en el medio** — revisar, editar, aprobar. **Por qué:** aportan lo que los CLI no dan (UI rica, diffs visuales) y ya hablan MCP. **Señal:** el mismo hub MCP consumido desde la app de ventana que preferís para revisar.

B6 La voz como disparador de orquestación (ya construida, sin explotar)

Tesis: el costo real de delegar es *arrancar*: abrir el panel, escribir el pedido, aprobar. Tu voz ya resuelve dictado, aprobación hablada y guion de pasos. Hipótesis: «che, que opencode arregle los warnings» debería crear el nodo de tarea, lanzar el worker y pedirte aprobación al terminar. **Señal:** una tarea de código iniciada sin tocar el teclado.

Nivel C · apuestas que cambian el juego (90 días o más)

C1 NodeFlow como infraestructura, no sólo como app

Tesis: si el valor está en el grafo + la bóveda + la evidencia, el crecimiento viene de que *otros orquestadores* usen el lienzo. Tu servidor MCP ya es el vehículo: publicarlo con 3 herramientas limpias (crear nodo, enlazar, leer contexto) te pone en el mismo carril que `modelcontextprotocol/servers` (90k★) pero con algo que casi nadie tiene: memoria visual del proyecto.

C2 MoA y el validador en código: calidad sin modelos más grandes

Tesis: tu regla de la casa (el modelo propone, el código valida) es exactamente lo que le falta a la industria. Hermes ya soporta `moa` (varias opiniones) y `fallback` (cadena de proveedores). Combinado: N modelos opinan → un validador en código decide → se registra la evidencia en el lienzo. Es más barato que un modelo frontera y da trazabilidad.

C3 Local-first y control de costo como producto de servicios

Tesis: tenés dos activos que las agencias no pueden copiar: **todo corre en la máquina del cliente** (privacidad demostrable) y **el costo medido por tarea** (contador de costo + caché semántica + ruteo). Eso no es una feature de la app: es la propuesta de un servicio («te dejo un sistema que no manda tus datos a nadie y te dice cuánto gasta»).

C4 El ecosistema como imán de leads, no como proyecto infinito

Tesis (incómoda): el ecosistema de agentes es infraestructura *interna*. Lo que se vende es el resultado: visuales, automatizaciones, y NodeFlow como demostración de criterio. Si el ecosistema se convierte en el proyecto, el imán deja de atraer porque nadie ve nada.

5 · Lo que no estamos teniendo en cuenta

#	Punto ciego	Por qué importa (y qué hacer)
1	La atención como recurso escaso	Seis clientes MCP, cinco apps de agente, tres CLIs: el costo no es la factura, es el ruido. Hace falta <i>un</i> panel (el lienzo) y <i>un</i> log de qué hizo cada worker.
2	Permisos de los agentes	Un worker con escritura en el repo y acceso al llavero es un riesgo real: ya pasó (un bot dejó NodeFlow con ventana blanca). Regla: cada worker con alcance declarado, worktree propio y revisión por tests antes de mergear.
3	Las cuotas como variable de diseño	Medido hoy: Grounding de Gemini agotado y GitHub Models retirado. Todo rol debe declarar su cuota, su límite y su plan B <i>antes</i> de asignársele trabajo.
4	16 GB de RAM	Ollama + Docker + 3 workers + navegador con 40 pestañas no entran. El paralelismo tiene techo físico: conviene cola explícita, no «lanzar todo».
5	Un solo modelo grande por GPU	Con 12 GB y carga concurrente, dos workers con modelos locales se pisan. Los workers locales deben serializarse; los cloud pueden paralelizar.
6	Bus factor y colaboración	Sin <code>AGENTS.md</code> , sin demo pública y sin instalador firmado, nadie más puede aportar — y vos quedás como único punto de falla.
7	Licencias	AutoGen es CC-BY-4.0; varios modelos <code>-cloud</code> tienen términos comerciales. Si NodeFlow va a distribuirse (MIT), la matriz de licencias de modelos y dependencias hay que mirarla ahora, no al vender.
8	Observabilidad de agentes	Tenés costo por llamada de la app, pero no por worker. Sin métrica por worker no hay decisión sobre cuál mantener.
9	Kùzu sin actividad	La capa de grafo es la fundación del producto y su repo lleva 11 meses sin commits. Encapsular el acceso ya.
10	El deadline real	El video demo del hackathon y los 30 prospectos cierran el 30/09. El ecosistema es medio, no fin: si se come la semana, el costo de oportunidad es todo el ciclo de ingresos.

6 · El camino que multiplica (mi recomendación)

Tres movimientos, en este orden, cada uno con su criterio de corte. No son «adoptar frameworks»: son *conectar lo que ya está instalado y poner una regla de verificación*.

Movimiento 1 · Hermes + opencode con contrato (esta semana)

Escribir `AGENTS.md` en `nodeflow-desktop/` y dejar una skill de Hermes (`delegar-a-opencode`) con el protocolo: *worktree aparte* → `opencode run` con tope → `cargo test` / `tsc` como árbitro → PR con `gh`. **Corte:** si en 5 días no resolvió 3 tareas reales sin que intervengas, el protocolo está mal escrito (no el worker).

Movimiento 2 · Un hub MCP único montado en todos los clientes (2 semanas)

NodeFlow (ya está) + navegador (`playwright-mcp`) + documentos (`markdown`). Montados en Hermes, opencode, Cline y Claude Desktop. Efecto lateral buscado: *empezás a usar la herramienta correcta para cada cosa en vez de una sola para todo*. **Corte:** si una capacidad no se usa desde dos clientes distintos, no la montes.

Movimiento 3 · Continuidad automática (antes del 30/09, 1 tarde)

Cron diario en Hermes: `git log` + DEVLOG + nodos PENDIENTE del lienzo → tres movimientos + un recordatorio de lo cortado. Sale en el lienzo (nodo) y en el panel. **Corte:** si a la semana no cambió ninguna decisión por esa lista, el formato está mal, no la idea.

6.1 Roadmap con criterios de corte

Ventana	Qué se hace	Señal de que sigue	Señal de que se corta
0-10 días antes del 30/09	Video demo + one-pager + 30 prospectos (el plan activo). Movimiento 1 y 3 en paralelo, sólo con ratos muertos.	Demo publicada y primeros contactos enviados.	El ecosistema consumió más de 3 días de la semana.
10-30 días	Movimiento 2 (hub MCP) + <code>AGENTS.md</code> maduro + métricas por worker (tokens y costo por tarea, tomando <code>opencode stats</code>).	Dos clientes usando el mismo hub; costo por tarea visible.	Más de un mes montando servidores MCP sin una tarea real resuelta.
30-60 días	B1 (Dagu o Windmill para flujos con reintentos) + B2 (puerta única al grafo) + B3 (un segundo agente-CLI de otra familia).	Un flujo largo reanudado solo; una migración de grafo hecha en un día.	Dos motores de flujo instalados y ninguno en producción.
60-90 días	Firma Authenticode + release público con <code>AGENTS.md</code> , y el MCP de NodeFlow publicado (C1).	Alguien externo (o un agente ajeno) opera el lienzo con tu MCP.	Publicar sin que nadie lo haya pedido.

6.2 Lo que NO haría (y por qué)

- **Meter LangGraph/CrewAI/agno en NodeFlow.** Ninguno vive en Rust ni piensa como app de escritorio: te deja un segundo runtime (Python) que hay que empaquetar, firmar y actualizar. Las ideas se copian en 200 líneas; la dependencia no se paga.
- **Instalar los siete orquestadores.** Los proyectos con 1★ y 3★ de la \$0 son experimentos ajenos: adoptarlos es heredar su bus factor, no resolver el tuyo.

- **Automatizar por UI las apps que no tienen CLI.** Es la clase de integración que se rompe en cada actualización y que ya te costó una app en blanco.
- **Comprar «la plataforma de agentes».** Ya tenés el orquestador más capaz de tu máquina: el trabajo es darle workers, memoria y continuidad, no reemplazarlo.

7 · Fuentes (verificadas el 19/09/2026)

Repos consultados con `gh api repos/<owner>/<repo>` (estrellas, licencia, último push). Especificaciones: MCP [modelcontextprotocol](#) · ACP [zed-industries/agent-client-protocol](#) · AGENTS.md [openai/agents.md](#). Retiro de GitHub Models: [github.blog/changelog/30/07/2026](#). Superficies locales: salida de `--help` de `opencode 1.18.18`, `aider`, `hermes`, y búsqueda de ejecutables en `%LOCALAPPDATA%\Programs`. Medición de cuotas: llamadas directas a la API de Gemini y a `models.github.ai` desde esta máquina.

NodeFlow · informe de ecosistema de agentes. Tres investigaciones delegadas en paralelo (modelo `gpt-oss:120b-cloud`) + verificación propia. Las afirmaciones sin marca de verificación son hipótesis de trabajo, no hechos. Generado el 19/09/2026 para TOMAS.WAV.